

Lab 08 - Invoice exception agent

AI Agents for Business | TGS-2023018987 | v1.0

Objective

Perform deterministic three-way matching and explain exceptions with evidence.

Alignment: K1, B19, B20. Scheduled hands-on time: 30 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- invoices.csv
- purchase-orders.csv
- receipts.csv

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Open the three CSVs and join them on po_id and vendor_id. Preserve invoice IDs so duplicate and missing-record cases stay traceable.
2. Run `python3 run.py`. Independently calculate INV-100: 12 units at 25 equals 300; only 10 units received gives a 50 difference.
3. Inspect the duplicate INV-100 row and the invoice with no matching PO. Confirm neither is silently marked payable.
4. Use prompt 1 to turn the deterministic results into an exception memo. Require invoice ID, relevant records, arithmetic, reason and owner.
5. Use prompt 2 to challenge a request to ignore a small mismatch. The agent should apply the stated policy rather than invent a tolerance.
6. Save learner-output.json and an approval checklist. The lab does not connect to accounting software or release payment.

Acceptance checks

- INV-100 variance is 50.00.
- Duplicate and missing-PO records are flagged.
- The memo does not authorise payment.

Run `python verify.py` after `run.py` (use `python3` if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's requirements.txt.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.

- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after assessment; keep source data and prompts unchanged.